Before we jump into experimenting with geospatial queries, we need to understand the mapping schema of geospatial data: the data types that support geodata and the mechanics for indexing that data. In the next section, we examine the `geo_point` type first and then `geo_shape`.

12.2 Geospatial data types

Similar to how the `text` data type represents textual data, Elasticsearch provides two dedicated data types to work with spatial data: `geo_point` and `geo_shape`. The `geo_point` data type expresses a longitude and latitude and works on location-based queries. The `geo_shape` type, on the other hand, lets us index geoshapes such as points, multi-lines, polygons, and a few others. Let's look at these spatial data types.

12.2.1 The geo_point data type

A location on a map is expressed universally by longitude and latitude. Elasticsearch supports representing such location data using a dedicated `geo_point` data type. We looked at the `geo_point` data type briefly in chapter 4; let's recap how we can define a field as a `geo_point` in our mapping schema. Once the mapping is ready, we can index a document. The following listing creates a data schema for a `bus_stops` index with a couple of fields.

Listing 12.1 Creating a mapping with `geo_point`

```
PUT bus_stops
{
  "mappings": {
    "properties": {
      "name":{
        "type": "text"
      },
      "location":{
        "type": "geo_point"
      }
    }
  }
}
```

Defines the location attribute as a geo_point data type

The `bus_stops` index is defined with two properties: `name` and `location`. The `location` property is represented by a `geo_point` data type, which means it should be set with latitude and longitude values when indexing the document. The following query indexes the London Bridge Station bus stop.

Listing 12.2 Indexing a bus stop with a location defined as a string

```
POST bus_stops/_doc
{
  "name":"London Bridge Station",
  "location":"51.07, 0.08"
}
```

Inputs the location as a string with latitude and longitude values